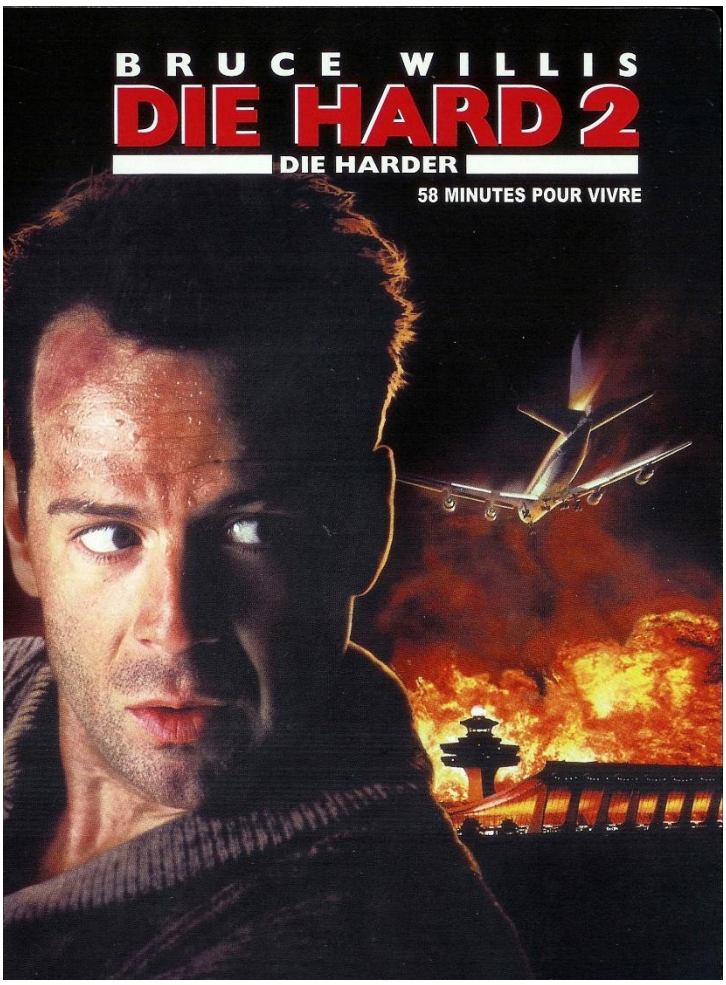
58 minutes pour vivre

Un Apprentissage Par Problème (APP) destiné aux étudiants du module Algorithmique avancée 1



Henri HAGBE

Eléments de contexte

L'aéroport de Roissy est sous la menace d'une tempête exceptionnelle. À 19h42, deux pistes viennent d'être fermées et le trafic entrant est maintenu en attente. Le centre de régulation doit prendre des décisions rapides et justifiables : **chaque minute compte**.

- **Établir un ordre d'atterrissage prioritaire à partir d'informations hétérogènes (carburant restant, urgence médicale, incident technique, importance diplomatique, heure d'arrivée prévue).**
- Concevoir un système de décision capable de s'adapter : changement de politique (POLICY) selon le contexte, stress tests, et réaction à un événement imprévu en séance retour.

Vous êtes mandatés comme équipe d'ingénieurs logiciel : votre objectif est de livrer un module fiable, modulaire et défendable en langage Python, qui permet de simuler l'impact de différents choix en situation critique. Voici les stress test classique auquel votre application doit être capable de répondre.

Il est 19h42.

Une tempête exceptionnelle frappe la région parisienne.

Deux pistes sont fermées. Une seule piste reste opérationnelle.

24 avions sont en approche.

Certains ont peu de carburant.

Certains signalent des incidents techniques.

Certains transportent des passagers en détresse médicale.

Certains ont une importance diplomatique élevée.

Le centre de contrôle vous confie une mission :

Concevoir un module informatique capable de déterminer l'ordre d'atterrissage des avions.

Vous avez 58 minutes avant que la situation ne devienne critique.

Travail demandé

Vous êtes une équipe d'ingénieurs en algorithmique, en charge d'un module de priorisation pour le contrôle aérien.
Votre mission : concevoir et développer en Python un système capable de classer des avions en approche selon une POLICY (politique d'accueil), puis d'en simuler les conséquences en situation de crise (carburant, incidents, contraintes temporelles).

Plus précisément, votre travail devra répondre aux exigences suivantes :

1. **Représentation des données et chargement du trafic :**
   - Charger le jeu de données initial fourni (liste de dictionnaires représentant les avions) et vérifier sa cohérence (champs attendus, valeurs, absence d'erreurs).

   - Implémenter des fonctions utilitaires (affichage, recherche d'un minimum, extraction d'un sous-ensemble) pour faciliter le débogage et la validation.

2. **POLICIES de priorisation (règles de décision) :**
   - Définir au moins trois POLICIES différentes (ex. priorité carburant, priorité incidents, priorité diplomatique), sous forme de comparateurs ou de fonctions de scoring.
   - Permettre de changer de POLICY sans réécrire l'algorithme de tri (séparation POLICY / ALGORITHME).

3. **Implémentation et comparaison des algorithmes de tri :**
   - Implémenter au moins deux tris en temps quadratique (tri par insertion et tri par sélection) pour établir l'ordre d'atterrissage.
   - Comparer empiriquement les algorithmes (nombre de comparaisons, éventuellement temps d'exécution) sur plusieurs tailles de trafic.

4. **Stress tests (tests de robustesse) :**
   - Programmer un générateur de trafic permettant de créer différents scénarios (normal, crise carburant, crise médicale, sommet diplomatique).
   - Tester votre module sur des volumes croissants (10, 30, 50, 100 avions) et analyser les limites du tri quadratique.

5. **Simulation dynamique et restitution :**
    - Simuler l'écoulement du temps : à chaque tour, un avion atterrit, le carburant des autres diminue, et un crash survient si fuel ≤ 0 !
    - Produire un bilan (avions sauvés / crashés) et justifier vos choix. **En séance retour, une contrainte imprévue sera ajoutée : votre système devra s'adapter rapidement. Les « tricheurs » seront foutus.**

Définition des Données

Le jeu de données décrit un ensemble d'avions en approche et les informations minimales nécessaires pour prioriser les atterrissages.

1. **Informations d'identification et de contexte**
    - **id :** Identifiant du vol (ex. "AF342").
    - **arrival_time :** Heure d'arrivée prévue (utile comme critère d'équité / tie-break).
    - **diplomatic_level :** Niveau d'importance diplomatique (1 à 5).
    - **(Optionnel) airline / origin :** Informations descriptives si vous souhaitez enrichir la simulation.
    - **(Optionnel) notes :** Champ libre pour annoter un événement (ex. "chef d'État", "cargo médical").

2. **Données de sécurité et de criticité**
    - **fuel :** Minutes de carburant restantes.
    - **medical :** Présence d'une urgence médicale à bord (booléen).
    - **technical_issue :** Incident technique / panne signalée (booléen).
    - **(Optionnel) fuel_burn_rate :** Surconsommation (ex. fuite) si vous modélisez une aggravation.
    - **(Optionnel) must_land :** Priorité absolue (utilisée lors de l'événement imprévu).
    - **(Optionnel) diverted :** Marqueur si un vol est dérouté plutôt qu'atterri.

Définition des POLICIES et de la priorisation

Une POLICY définit la règle de décision : elle transforme des informations hétérogènes en un ordre d'atterrissage.

Dans ce projet, vous devez séparer la POLICY (quoi prioriser) de l'algorithme de tri (comment classer).

Convention recommandée : un comparateur prend deux avions a1 et a2 et renvoie True si a1 doit être placé avant a2.

```
def policy(a1, a2):
    # True si a1 doit être avant a2
    ...
```

Exemple de hiérarchie de crise (à adapter) :

- Incident technique *(technical_issue*) prioritaire
- Urgence médicale (*medical*) prioritaire
- Carburant le plus faible (*fuel*) prioritaire
- Importance diplomatique (*diplomatic_level*) pour départager

En cas d'égalité, vous devez définir un tie-break (ex. heure d'arrivée) et discuter la stabilité de votre tri.

Le code doit être structuré de manière claire et modulaire, en respectant les bonnes pratiques de la programmation Python :

1. **Clarté des variables :** Utiliser des noms explicites et significatifs pour les variables et les fonctions (policy_fuel, tri_selection, simulation, ...).
2. **Organisation du code :** Diviser le code en fonctions bien définies (génération de trafic, tri, policies, simulation, analyse).
3. **Documentation :** Chaque fonction doit comporter une docstring expliquant son rôle, ses paramètres et ses valeurs de retour.
4. **Structure du projet :** Organiser le projet en plusieurs fichiers si nécessaire (ex. policies.py, tris.py, simulation.py, main.py).
5. **Utilisation de commentaires :** Ajouter des commentaires pertinents, notamment pour expliquer les choix de priorité (dimension "politique" de la policy).

Les ressources pour traiter la situation-problème

- Le cours/TD « Les algorithmes de tri 1 » (tri par sélection, tri par insertion, stabilité, complexité).
- Le cours « Langage Python » (listes, dictionnaires, fonctions, modules).
- Optionnel : bases de mesure de performance (time/perf_counter) et génération aléatoire (module random) pour le stress test.
- Le jeu de données initial (fourni) et les consignes de séance retour (événement imprévu).

Livrables attendus

- Le projet respectant les exigences ci-dessus (script Python ou notebook), incluant : policies, tris quadratiques, stress tests et simulation.
- Un jeu de tests (ou script de démonstration) permettant de reproduire vos résultats sur plusieurs scénarios.
- Un rapport synthétique (2 à 4 pages) expliquant :
    - La structure du projet et la répartition du travail dans l'équipe (rôles, contributions).
    - Les choix techniques : représentation des avions, policies, algorithmes de tri, métriques (comparaisons, temps).
    - Les difficultés rencontrées, la manière dont vous les avez résolues, et une analyse critique (limites du tri quadratique, stabilité).

Une présentation orale (5 à 7 minutes) en séance retour : démonstration, justification de la policy, et adaptation à l'événement imprévu.
Une attention particulière sera accordée au respect des bonnes pratiques de programmation et à la modularité (POLICY / ALGORITHME).

Objectifs d'apprentissage de l'APP (AAV)

- Manipuler des listes de dictionnaires en Python (recherche, filtrage, extraction, mise à jour).
- Comprendre et maîtriser les tris quadratiques (tri par sélection, tri par insertion) et la notion de stabilité.
- Tester et comparer empiriquement des algorithmes (comparaisons, temps d'exécution) et relier les résultats à la complexité $O(n^2)$.
- Collaborer efficacement en équipe : rôles, organisation, versionnement éventuel, et restitution claire.
- Concevoir un système décisionnel modulaire : séparation POLICY (règles) / ALGORITHME (tri) / SIMULATION (dynamique).
- Développer une capacité d'analyse critique : compromis "justice / sécurité / efficacité" et adaptabilité à une contrainte imprévue.
- Respecter les bonnes pratiques Python (noms, docstrings, structure, lisibilité) pour produire un code maintenable.